

Podstawowe dane

Imie i nazwisko	Robert Mesek Wojciech Kaźmierczak
Data	02.01.2024
Tytuł	Projekt z Algorytmów Geometrycznych
Numer projektu	7

Dane techniczne

Procesor	Intel i7-1165G7 x86_64
Język	Python 3.11.5
Biblioteki	numpy 1.25.2, pandas 2.0.3, matplotlib 3.7.2, notebook 6.5.4, sortedcontainers 2.4.0
System operacyjny	WSL Ubuntu 22.04 (MS Windows 10 22H2)
Środowisko	conda 23.11.0

Opis projektu

Należy zaimplementować algorytmy konstrukcji triangulacji dla dowolnego wielokąta prostego dwoma metodami. W obu przypadkach triangulacja ma się opierać jedynie na wierzchołkach wielokąta (bez dodawania wierzchołków wewnątrz wielokąta).

Pierwsza metoda to podział wielokąta na wielokąty monotoniczne (zgodnie z algorytmem podanym na wykładzie), a następnie triangulacja wynikłych z podziału wielokątów monotonicznych także metodą podaną na wykładzie (i realizowaną na ćwiczeniach).

Drugi algorytm to triangulacja Delaunay'a z ograniczeniami (z procedurami odzyskania krawędzi i usunięcia trójkątów zewnętrznych). W tej metodzie wybrać odpowiedni sposób poszukiwania trójkąta w istniejącej triangulacji (i go omówić).

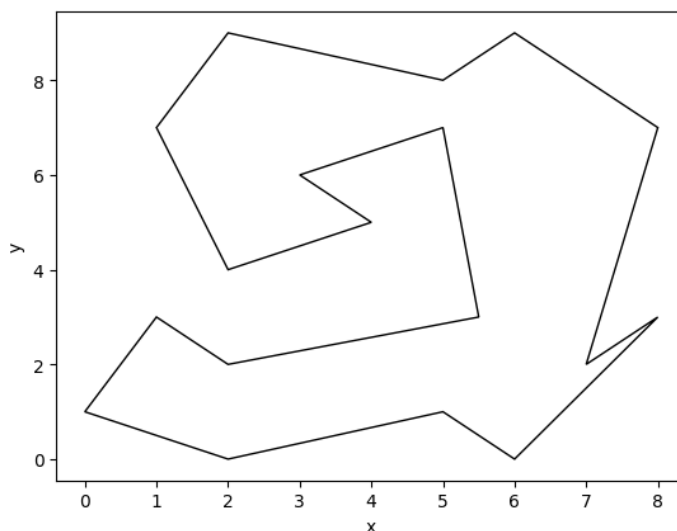
Triangulacja polegająca na podziale wielokątów

Idea algorytmu

Celem algorytmu jest znalezienie triangulacji dowolnego wielokąta prostego. Aby to osiągnąć w tym sposobie musimy przeprowadzić kilka kroków:

1. Klasyfikacja wierzchołków danego wielokąta.
2. Podział wielokąta na wielokąty monotoniczne.
3. Triangulacja wielokątów monotonicznych.

Wielokąt na którym przedstawimy ideę



Rys. 1: Przykładowy wielokąt prosty.

1. Klasyfikacja wierzchołków danego wielokąta

Typu wierzchołków

Wierzchołki naszego wielokąta możemy podzielić na parę kategorii:

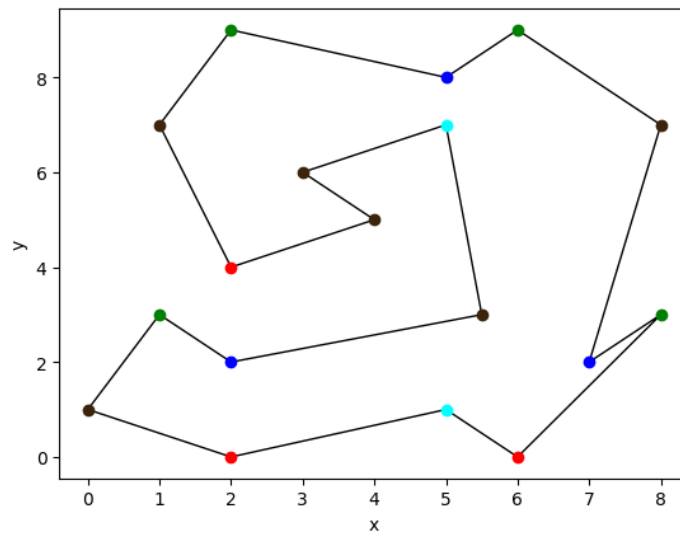
- początkowe, gdy obaj jego sąsiedzi leżą poniżej i kąt wewnętrzny ma mniej niż 180 stopni. To wierzchołki, w których zaczyna się monotoniczny spadek
- końcowe, gdy obaj jego sąsiedzi leżą powyżej i kąt wewnętrzny ma mniej niż 180 stopni. To wierzchołki, w których monotoniczność wielokąta się zmienia, czyli na przykład zaczyna się monotoniczny wzrost, jeśli wcześniej był spadek, lub na odwrót.
- dzielące, gdy obaj jego sąsiedzi leżą poniżej i kąt wewnętrzny ma więcej niż 180 stopni. To wierzchołki, które wyznaczają przekątne (linie łączące), tworzące trójkąty podczas triangulacji.
- łączące, gdy obaj jego sąsiedzi leżą powyżej i kąt wewnętrzny ma więcej niż 180 stopni. To wierzchołki, które są połączone liniami (przekątnymi) wewnątrz wielokąta, tworząc trójkąty.
- prawidłowe, pozostałe przypadki, jeden sąsiad powyżej, drugi poniżej

Wierzchołki łączące i dzielące odgrywają kluczową rolę w procesie triangulacji wielokątów, pozwalając na podział figury na trójkąty w sposób bezkolizyjny.

Oznaczenia wierzchołków kolorami

- Początkowy - kolor zielony
- Końcowy - kolor czerwony
- Łączący - kolor niebieski
- Dzielący - kolor cyjan
- Prawidłowy - kolor brązowy

Rezultat kolorowania



Rys. 2: Przykładowy wielokąt po klasyfikacji.

Wykorzystane funkcje

`classify(a: Point, b: Point, c: Point, eps) -> PointType`

Zwraca typ punktu, wykorzystując własność wyznacznika macierzy liczonego ze wzoru:

`det = (a.x - c.x) * (b.y - c.y) - (a.y - c.y) * (b.x - c.x)`

2. Podział wielokąta na wielokąty monotoniczne

Konwersja wielokąta na podwójnie łączoną listę krawędzi

Dla naszego algorytmu pomocne jest sporządzenie struktury, w której będziemy mieli możliwość szybkiego dostępu do elementów wielokąta takich jak: kolejne krawędzie wraz z punktem będącym jej pomocnikiem, punkty z informacją do jakiej krawędzi należą.

Rozwiązanie będzie korzystać z algorytmu zmiatania

W tym celu będziemy potrzebować 2 pomocniczych struktur danych:

- Q - struktura zdarzeń - kolejka priorytetowa - są to nasze punkty, gdzie największy priorytet ma górny lewy punkt
- T - struktura stanu - dynamiczne drzewo wyszukiwań binarnych - są to krawędzie posortowane od lewej strony do prawej w miejscu przecięć z miotłą

Przebieg algorytmu

Kolejne punkty z kolejki priorytetowej Q będziemy przetwarzać w sposób zależny od typu danego punktu. Głównym celem jest wyeliminowanie wierzchołków łączących oraz dzielących. Wielokąt y-monotoniczny to takich, który właśnie nie posiada żadnych wierzchołków tego typu.

W tym celu dla każdego wierzchołka dzielącego dodajemy przekątne wychodzące w górę, a dla każdego wierzchołka łączącego wychodzące w dół. Przekątne te mają nie przecinać żadnej innej przekątnej.

Będziemy starali się znaleźć wierzchołki znajdujące się w pobliżu rozpatrywanego aktualnie wierzchołka v_i . Aby szybko znajdować taki wierzchołek dla v_i dzielącego zauważmy, że będzie on najniższym wierzchołkiem leżącym pomiędzy prostą e_l będącą bezpośrednio na lewo od v_i oraz e_p będącą bezpośrednio po prawej stronie. Wierzchołek ten nazwiemy pomocnikiem(v_i), zauważmy, że może on być górnym końcem samej e_l .

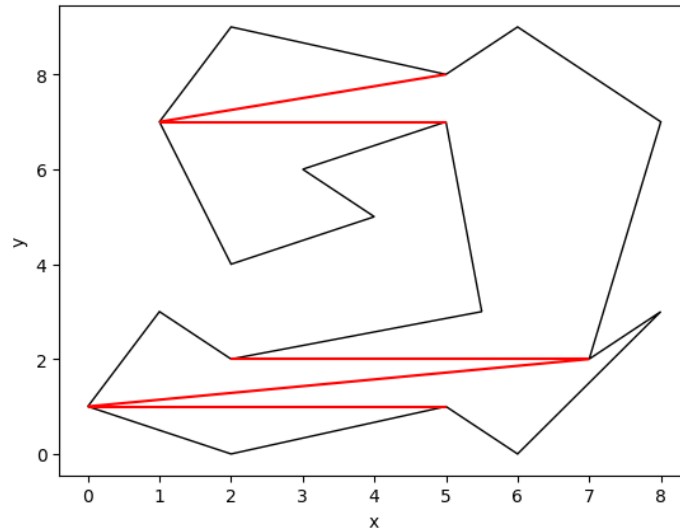
Pozostają jeszcze wierzchołki v_i łączące. Potrzebować one będą wierzchołków znajdujących się pod miotłą (jeszcze nie przetworzonych), ale będzie to dokładnie odwrócona sytuacja dla wierzchołka dzielącego. Za pomocnika(v_i), będziemy wybierać wierzchołek leżący pomiędzy e_l a e_p , znajdujący się najbliżej.

Dowód poprawności oraz wyprowadzenie

Dowód poprawności oraz szczegóły wyprowadzenia algorytmu znajdziemy w książce

[1] "Geometria obliczeniowa. Algorytmy i zastosowania" M. Berg

Rezultat podziału



Rys. 3: Przykładowy wielokąt po podziale.

Wykorzystane funkcje

`make_monotone(polygon) -> tuple[list[Edge], list[Edge]]`

Główna funkcja wykonująca algorytm zmiatania. Przyjmuje ona wielokąt w postaci listy kolejnych punktów w kierunku przeciwnym do ruchu wskazówek zegara. Zwraca ona listę krawędzi wielokąta oraz listę nowo dodanych wierzchołków.

`handle_start_vertex(T: SortedList, color_point: ColorPoint)`

Funkcja przetwarzająca wierzchołki początkowe.

`handle_end_vertex(T: SortedList, D: list[Edge], color_point: ColorPoint)`

Funkcja przetwarzająca wierzchołki końcowe.

`handle_split_vertex(T: SortedList, D: list[Edge], color_point: ColorPoint)`

Funkcja przetwarzająca wierzchołki dzielące.

`handle_merge_vertex(T: SortedList, D: list[Edge], color_point: ColorPoint)`

Funkcja przetwarzająca wierzchołki łączące.

`handle_regular_vertex(T: SortedList, D: list[Edge], color_point: ColorPoint)`

Funkcja przetwarzająca wierzchołki prawidłowe.

`get_points_and_diagonals(polygon: list[tuple[float, float]])`

Pomocnicza funkcja przetwarzająca wynik działania `make_monotone()` na listę współrzędnych punktów oraz krawędzi w postaci indeksów, które w punkty należy połączyć w liście współrzędnych.

3. Triangulacja wielokątów monotonicznych

Konwersja punktów i przekątnych na poszczególne wielokąty y-monotoniczne

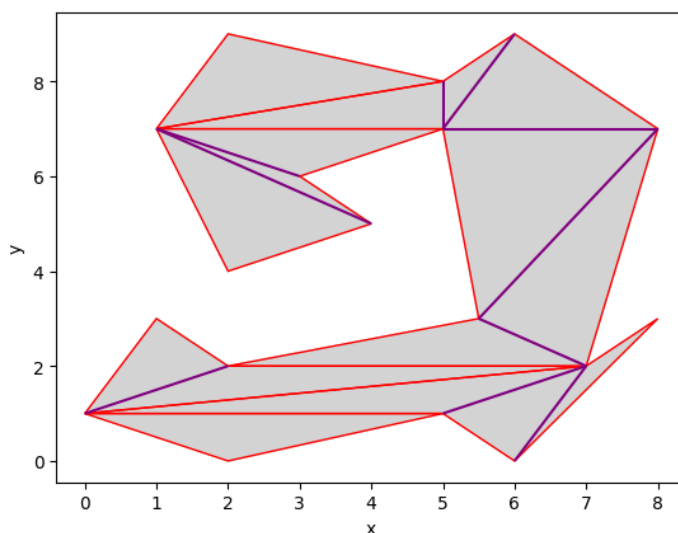
Sporządzimy listę punktów, jako strukturę gdzie każdy punkt posiada zbiór innych punktów z których możemy do niego dotrzeć na podstawie otrzymanych przekątnych.

Posłużymy się następnie lekko zmodyfikowanym algorytmem przeszukiwania wzdłuż. Dla każdego punktu będziemy szukać ścieżek w których wracamy do punktu startu. Jedyne na co musimy uważać, to aby od razu nie wrócić się do tego samego punktu. Po znalezieniu takiej ścieżki usuwamy połączenia ze struktury i próbujemy dla kolejnego punktu.

Triangulacja kolejnych wielokątów y-monotonicznych

- Określamy lewy i prawy łańcuch wielokąta względem kierunku monotoniczności
- Porządkujemy wierzchołki wzdłuż kierunku monotoniczności
- Wkładamy dwa pierwsze wierzchołki na stos.
- Jeśli kolejny wierzchołek należy do innego łańcucha niż wierzchołek stanowiący szczyt stosu, to możemy go połączyć ze wszystkimi wierzchołkami na stosie. Na stosie zostają dwa wierzchołki, które były „zamiatane” ostatnie.
- Jeśli kolejny wierzchołek należy do tego samego łańcucha co wierzchołek ze szczytu stosu, to analizujemy kolejne trójkąty, jakie tworzy dany wierzchołek z wierzchołkami zdejmowanymi ze stosu:
 - Jeśli trójkąt należy do wielokąta, to dodajemy przekątną i usuwamy odpowiedni wierzchołek ze szczytu stosu
 - w przeciwnym przypadku umieszczamy badane wierzchołki na stosie.

Rezultat triangulacji



Rys. 4: Przykładowy wielokąt po triangulacji.

Wykorzystane funkcje

`triangulate_y_monotone(polygon: list[tuple[float, float]])`

Główna funkcja, która dla znalezionej wielokąta y-monotonicznego znajdzie zwróci jego przekątne, które należy dodać w postaci współrzędnych końców przekątnych.

`find_chains(points: list[Point]) -> tuple[set[int], set[int]]`

Funkcja tworząca łańcuch lewy i prawy.

`triangle_in_polygon(chain: set[int], a: Point, b: Point, c: Point)`

Funkcja sprawdzająca czy trójkąt znajduje się wewnątrz wielokąta, korzysta z wyznacznika macierzy.

`check_same_chains(left_chain: set[int], right_chain: set[int], p1: Point, p2: Point)`

Funkcja sprawdzająca czy dwa punkty znajdują się na tych samych czy różnych łańcuchach.

`check_neighbours(points: list[Point], a: Point, b: Point)`

Funkcja sprawdzająca czy punkty są dla siebie sąsiadami.

Kompletny algorytm

Dane wejściowe

Wielokąt prosty zadany w postaci punktów w kierunku przeciwnym do wskazówek zegara.

Rezultat

Triangulacja wielokąta prostego w postaci przekątnych, które należy dołożyć.

Złożoność czasowa

Podział wielokąta o n wierzchołkach na wielokąty y-monotoniczne: czas $O(n \cdot \log(n))$, pamięć $O(n)$

Triangulacja y-monotonicznego wielokąta o n wierzchołkach $O(n)$

Złożoność czasowa kompletnego algorytmu dla n wierzchołków: $O(n \cdot \log(n))$

Złożoność pamięciowa kompletnego algorytmu dla n wierzchołków: $O(n)$

Bibliografia

- [1] "Geometria obliczeniowa. Algorytmy i zastosowania" M. Berg M. Kreveld M. Overmars
ISBN: 9788320432442 WNT Warszawa 2007

Triangulacja Delaunay'a z ograniczeniami

Program

Ścieżka do programu to: `scripts/helper/triangulate_delaunay.py`

Zaimportowane biblioteki

- `functools`
- `math`
- `operator`
- `matplotlib`
- `queue`
- `Visualizer()` z pliku `./scripts/visualizer/main`

Użyte struktury

`Point` (reprezentuje punkt w R^2):

- `x` – współrzędna x
- `y` – współrzędna y
- `edges` – zbiór półkrawędzi, których początkiem jest dany punkt

`Half_Edge` (reprezentuje półkrawędź):

- `origin` – wierzchołek początkowy (obiekt klasy `Point`)
- `next` – następnik (obiekt klasy `Half_Edge`)
- `prev` – poprzednik (obiekt klasy `Half_Edge`)
- `face` – ściana, której jest częścią
- `twin` – półkrawędź leżąca na tej samej krawędzi lecz skierowana przeciwnie

`Triangle` (reprezentuje trójkąt, ścianę siatki):

- `edges` – zbiór krawędzi (typ `Half_Edges`), którymi jest otoczony, skierowane przeciwnie do ruchu wskazówek zegara
- `visited` – pole do oznaczenia, iż dany trójkąt został już odwiedzony w danej iteracji

Delaunay (klasa reprezentująca triangulację Delaunay'a):

- `triangulation_faces` – przechowuje zbiór obiektów `Triangle` będących częścią obecnej triangulacji ruchu wskazówek zegara
- `Points` – tablica obiektów `Point` przechowująca punkty wielokąta
- `middle` – środkowy punkt trójkąta zawierającego wszystkie punkty zbioru
- `starting_points` – tablica punktów startowych, będących wierzchołkami trójkąta zawierającego wszystkie punkty zbioru
- `middle_triangle` – obiekt typu `Triangle` reprezentujący trójkąt znajdujący się na środku (trójkąt zawierający punkt środkowy)

Klasy Point, Half_Edge oraz Triangle posiadają zdefiniowane metody `__hash__`, `__eq__`, oraz funkcje zwracającą daną strukturę kolejno: jako krotkę współrzędnych; krotkę punktów reprezentowanych jako krotki współrzędnych; krotkę półkrawędzi reprezentowanych w sposób opisany wyżej.

Funkcje nie zawarte w żadnej z klas:

`orient(Half_Edge, Point)` funkcja zwraca liczbę, jeśli jest ona dodatnia to punkt przyjęty w argumentach leży po lewej stronie półkrawędzi (przyjętej w argumentach), jeśli ujemna to po prawej, a zero na linii zawierającej daną półkrawędź zwraca liczba wyliczana jest na podstawie wyznacznika:

$$\text{orient}(\text{Half_Edge } e, \text{Point } p) = \begin{vmatrix} e.\text{origin}.x & e.\text{origin}.y & 1 \\ e.\text{twin}.\text{origin}.x & e.\text{twin}.\text{origin}.y & 1 \\ p.x & p.y & 1 \end{vmatrix}$$

`is_point_circumcircle(Triangle, Point)` – zwraca wartość bool jeśli dany punkt zawiera się w trójkącie opisanym na trójkącie

`circumcircle(Triangle)` – zwraca współrzędne środka okręgu opisanego na trójkącie i jego promień

`create_edge(Point p1, Point p2, True/False)` – tworzy krawędź, gdy wartość True to dodaje do zbiorów krawędzi której dany punkt jest częścią jeśli False to nie

`connect_edge(Half_Edge first, Half_Edge scnd)` – łączy półkrawędzie ustawiając ich next i prev

`vis_triangulation()` – funkcja tworząca wizualizację triangulacji

`draw_polygon_by_clicking()` – pozwala na wprowadzenie wielokąta myszką

`if_point_in_triangle(Triangle, Point)` – sprawdza czy dla każdej półkrawędzi trójkąta punkt leży po lewej stronie, jeśli tak zwraca True jeśli nie False

`get_intersecting_edge(Triangle, Point p, Point p_next)` – jeśli jakaś krawędź danego trójkąta przecina się z krawędzią p, p_next to zwraca tą krawędź, do orzeczenia czy się przecinają używa funkcji orient, dla danej półkrawędzi końce drugiego odcinka muszą leżeć po jej obu stronach(jeden po tej drugi po tej) analogicznie zamieniając role odcinków (półkrawędzi).

Funkcje klasy Delauney:

`get_super_triangle()` – przyjmuje tablicę punktów na jej podstawie wylicza współrzędne wierzchołków trójkąta zawierającego wszystkie punkty ze zbioru oblicza $\max_x$, $\max_y$, $\min_x$, $\min_y$, będące minimalnymi/maksymalnymi współrzędnymi x/y na ich podstawie oblicza współrzędne środka $((\max_x + \min_x)/2, (\max_y + \min_y)/2)$ oraz delte będącą maksymalną różnicą z różnic $\max_x - \min_x$ oraz $\max_y - \min_y$, współrzędne punktów będący wierzchołkami „super trójkąta” to $(\text{mid_x} - 2 * \text{delta}, \text{mid_y} - \text{delta})$, $(\text{mid_x}, \text{mid_y} + 2 * \text{delta})$ oraz $(\text{mid_x} + 2 * \text{delta}, \text{mid_y} - \text{delta})$ mamy wtedy pewność że wszystkie punkty zbioru zawierają się w tym trójkącie.

`create_triangle()` – tworzy Triangle na podstawie wczytanych obiektów typu Half_Edges

`update_triangle()` – funkcja odpowiedzialna za wyszukanie trójkątów które w swoich okręgach opisanych zawierają dany punkt, odbywa się to za pomocą algorytmu BFS

`execute_triangulation_update()` - usuwa trójkąty wyszukane w `update_triangle()`

`get_triangle_in_which_the_point_is(Point)` - zwraca trójkąt, w którym zawiera się punkt. Zaczyna od trójkąta środkowego i sprawdza czy jest w nim punkt za pomocą funkcji `orient` jeśli tak zwraca dany trójkąt, jeśli `orient` zwrócił liczbę ujemną to przechodzi do trójkąta sąsiadującego właśnie tą krawędzią i szuka w nim itd.

`add_point()` – funkcja dodająca punkt wywołuje `get_triangle_in_which_the_point_is()`, `update_triangle()`, `execute_triangulation_update()` po czym tworzy nową triangulację na obszarze który został zakwalifikowany do zmiany i z którego usunięto krawędzie, następuje przepięcie wskaźników.

`add_points()` – to funkcja, która dla każdego punktu (wierzchołka wieloboku) wykonuje funkcję `add_point()`

`recover_the_edges()` – funkcja odpowiedzialna za zmianę istniejącej triangulacji tak by zawierała krawędzie wieloboku, w tym celu dla każdej krawędzi wieloboku sprawdza czy znajduje się ona w triangulacji jeśli nie to przesuwając wirtualnie punkt o drobny odcinek w celu uniknięcia złego wyboru trójkąta w którym się znajduje wykonuje funkcję `get_intersecting_edge()` i modyfikuje triangulację by ta zawierała krawędź wielokąta oraz pozostała triangulacją

`delete_triangles_not_in_polygon()` – przyjmując jako pierwotny trójkąt którego częścią jest półkrawędź od pierwszego do drugiego punktu z tablicy punktów wieloboku (mamy pewność, że ten trójkąt zawiera się w wieloboku, bo krawędzie trójkątów są zorientowane przeciwnie do wskazówek zegara tak jak kolejność punktów w tablicy wierzchołków wieloboku) po czym za pomocą algorytmu BFS przechodzimy po sąsiadach danego trójkąta dodając je do kolejki tylko, wtedy gdy między trójkątem pierwotnym, a sąsiadem nie ma krawędzi wieloboku(w ten sposób poruszamy się ciągle wewnątrz wieloboku. Wszystkie odwiedzone krawędzie są częścią ostatecznej triangulacji, zatem wszystkie inne zostają usunięte.

`get_delaunay_triangulation()` wywołuje po kolei funkcje `add_points()` w celu utworzenie wstępnej triangulacji, `recover_the_edges()` w celu uzgodnienia krawędzi, oraz `delete_triangles_not_in_polygon()` w celu usunięcia zbędnych trójkątów.

Użytkownik

Ścieżka do programu to: `scripts/helper/triangulate_delaunay.py`

Aby program zwrócił triangulację Delaunay'a należy podać listę krotek reprezentujących wierzchołki wielokąta podane w kolejności przeciwnej do ruchu wskazówek zegara. Inicjalizujemy triangulację tworząc obiekt `Delaunay(lista_wierzchołków)` po czym na tym obiekcie wykonujemy `get_delaunay_triangulation(True)` jeśli z wizualizacją `False` wpp)

Opis problemu

Triangulacja Delaunaya wielokąta prostego to przypadek triangulacji Delaunaya z ograniczeniami.

Oznacza to założenia:

- Nie można usunąć ani dodać wierzchołka, jeśli chcemy dokonać zmiany trzeba uruchomić program od początku.
- Każda krawędź z wejściowego wielokąta musi być reprezentowana w końcowej triangulacji.
- Poprzez ograniczenia czasami może nie zostać spełniony warunek nie zachodzenia na siebie obcych punktów do okręgu trójkąta, jednak ze względu na drugie ograniczenie nie dodawania nowych punktów nie powinno generować to błędów

Trzy podstawowe etapy algorytmu

WSTĘPNA TRIANGULACJA

Algorytm tworzenia wstępnej triangulacji działa na bazie rekurencyjnej:

- Mając triangulację n -elementową, dokładamy punkt P tworząc triangulację $n+1$ elementową poprzez znalezienie wszystkich trójkątów, których okrąg opisany zawiera dodawany punkt. Następnie każdy z tych trójkątów zostaje usunięty po czym dodany punkt jest łączony z wszystkimi wierzchołkami mini wieloboku który powstał z usunięcia błędnych ścian siatki triangulacji.
- Aby algorytm za każdym razem miał pełną triangulację na początku, stworzone są tymczasowe wierzchołki zewnętrzne, które okrywają dużym trójkątem cały wielokąt. Na końcu zostają one usunięte, na etapie usuwanie trójkątów.

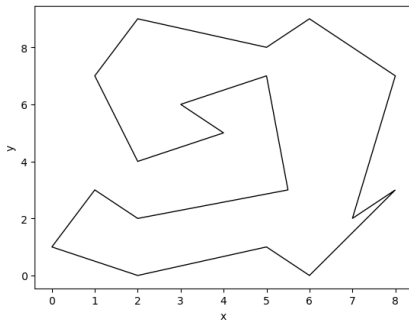
ODZYSKIWANIE KRAWĘDZI

Następnie przystępujemy do odzyskania krawędzi, poprzez zamienianie przekątnych w parach trójkątów, których obecne przekątne przecinają pierwotne boki wielokąta.

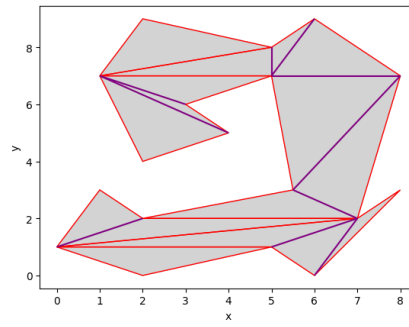
USUWANIE TRÓJKĄTÓW ZEWNĘTRZNYCH

Końcowym etapem jest usunięcie wszystkich trójkątów niezawierających się w danym wielokącie.

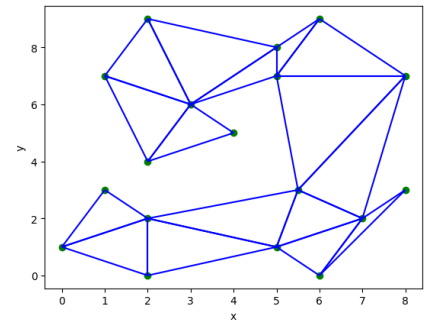
Porównanie algorytmów



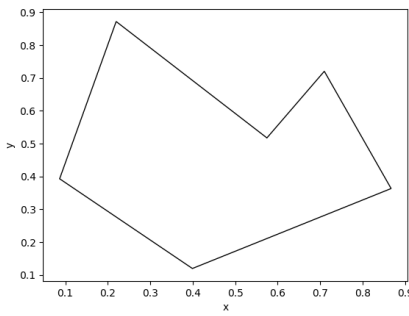
Rys. w0 Wielokąt



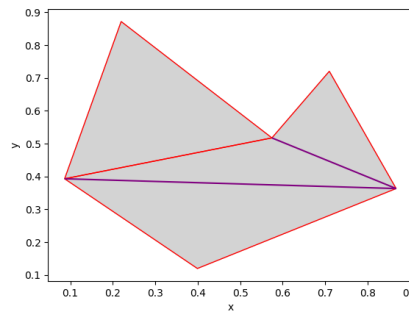
Rys. p0 Triangulacja podział



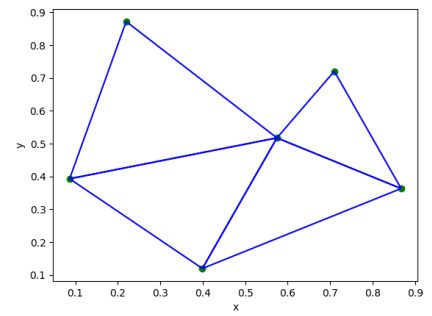
Rys. d0 Triangulacja Delaunay'a



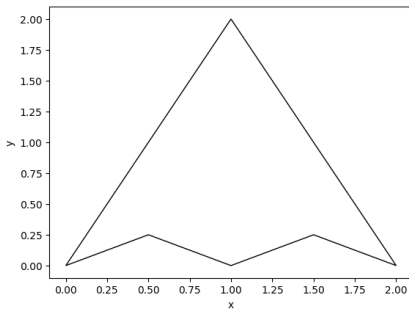
Rys. w1 Wielokąt



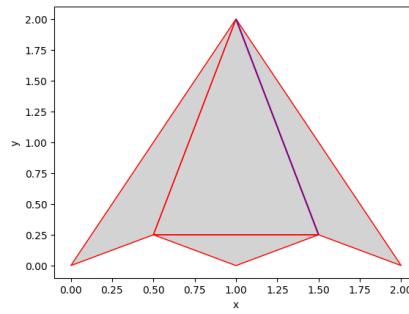
Rys. p1 Triangulacja podział



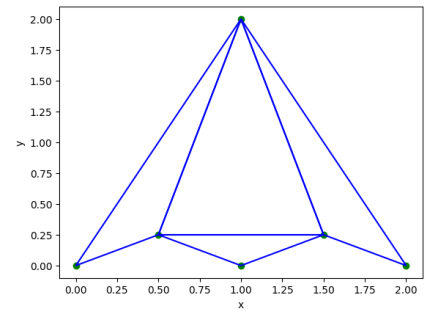
Rys. d1 Triangulacja Delaunay'a



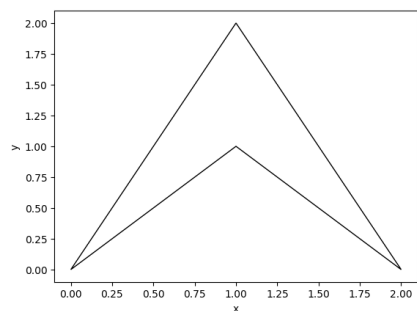
Rys. w2 Wielokąt



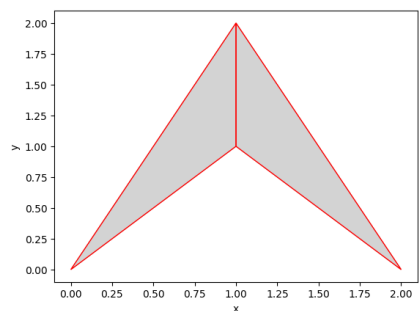
Rys. p2 Triangulacja podział



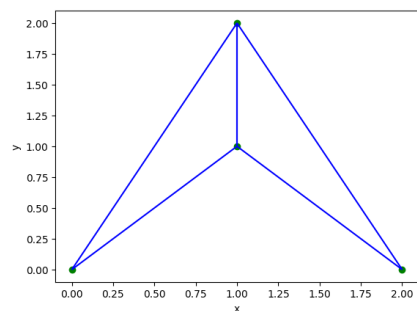
Rys. d2 Triangulacja Delaunay'a



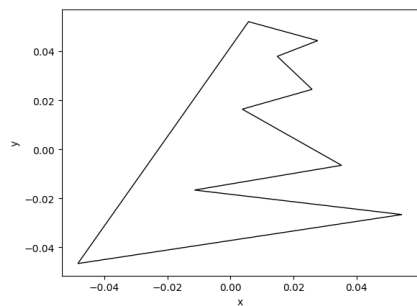
Rys. w3 Wielokąt



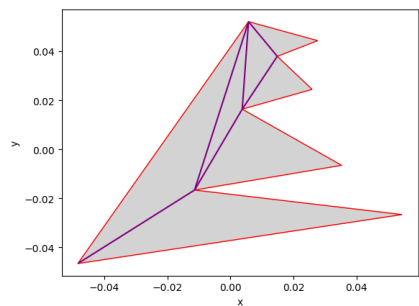
Rys. p3 Triangulacja podział



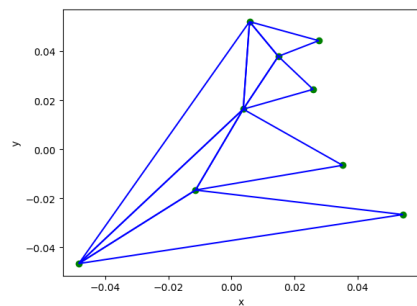
Rys. d3 Triangulacja Delaunay'a



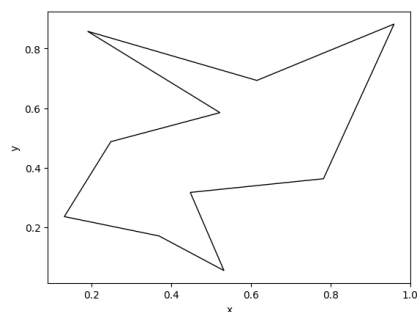
Rys. w4 Wielokąt



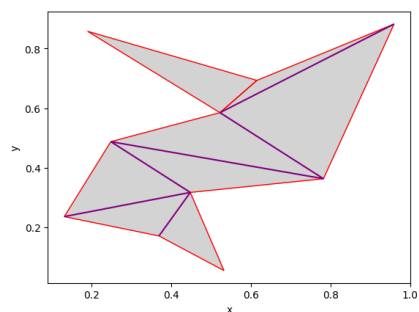
Rys. p4 Triangulacja podział



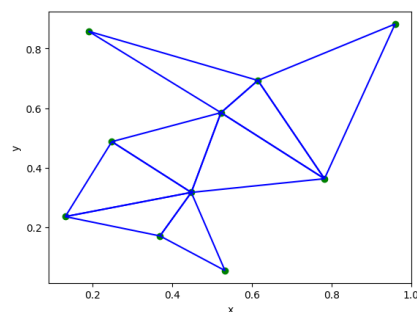
Rys. d4 Triangulacja Delaunay'a



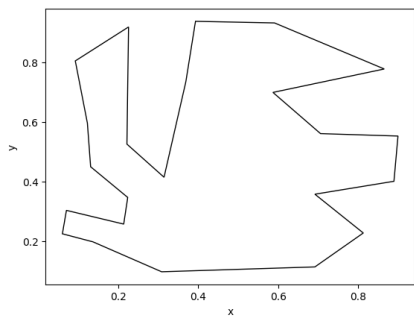
Rys. w5 Wielokąt



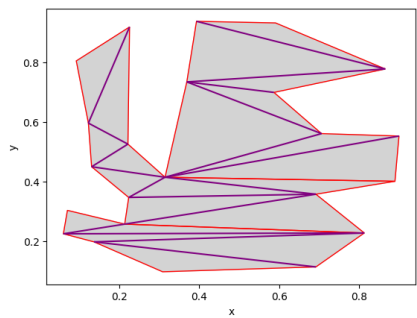
Rys. p5 Triangulacja podział



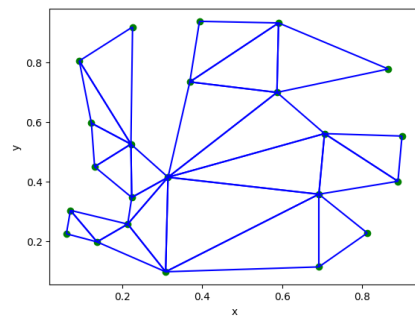
Rys. d5 Triangulacja Delaunay'a



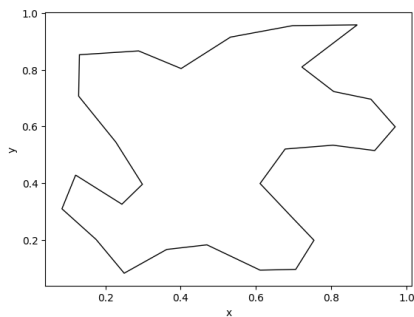
Rys. w6 Wielokąt



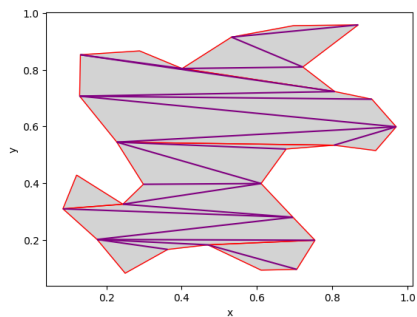
Rys. p6 Triangulacja podział



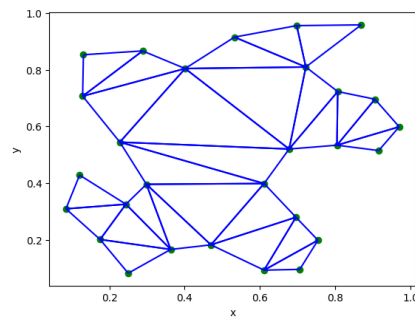
Rys. d6 Triangulacja Delaunay'a



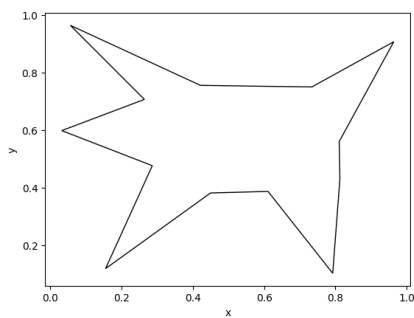
Rys. w7 Wielokąt



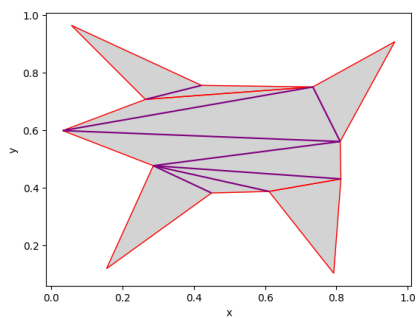
Rys. p7 Triangulacja podział



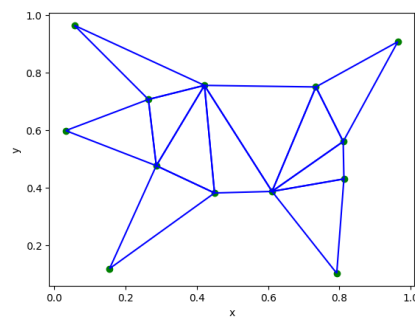
Rys. d7 Triangulacja Delaunay'a



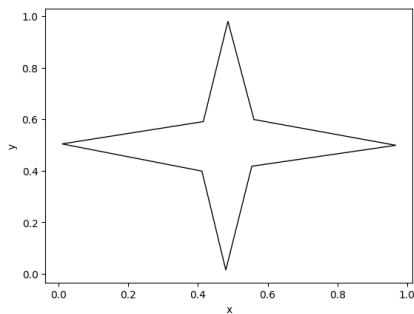
Rys. w8 Wielokąt



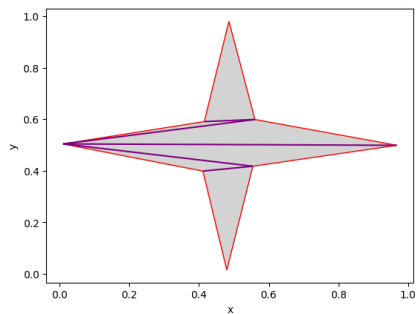
Rys. p8 Triangulacja podział



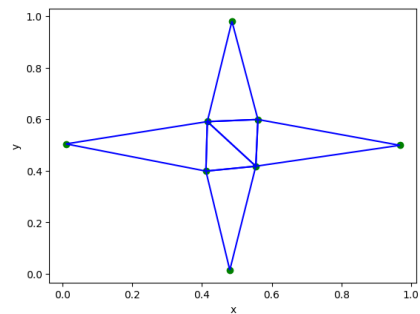
Rys. d8 Triangulacja Delaunay'a



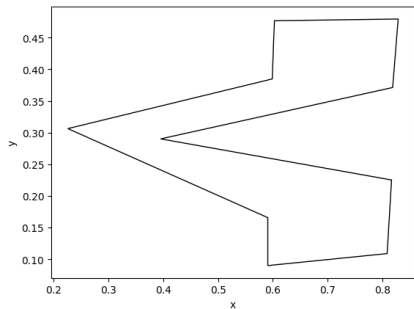
Rys. w9 Wielokąt



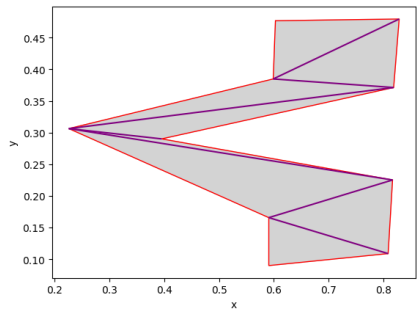
Rys. p9 Triangulacja podział



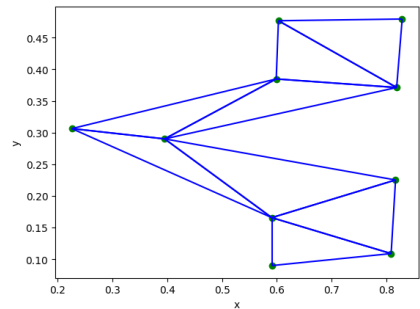
Rys. d9 Triangulacja Delaunay'a



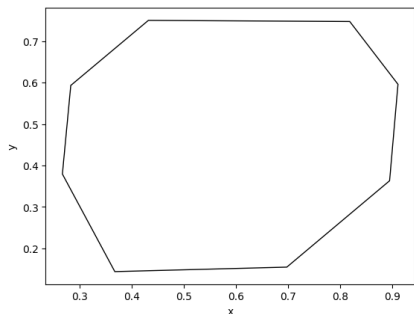
Rys. w10 Wielokąt



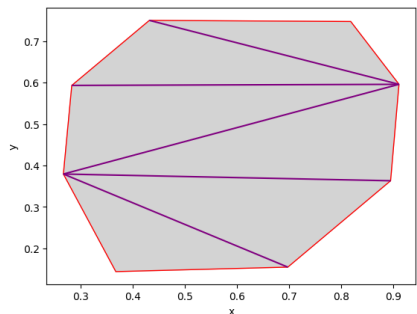
Rys. p10 Triangulacja podział



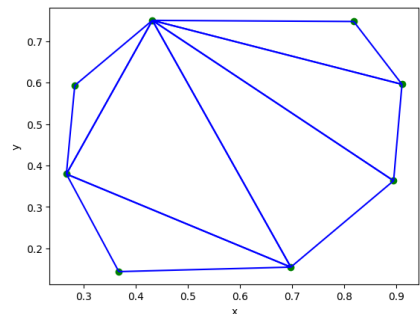
Rys. d10 Triangulacja Delaunay'a



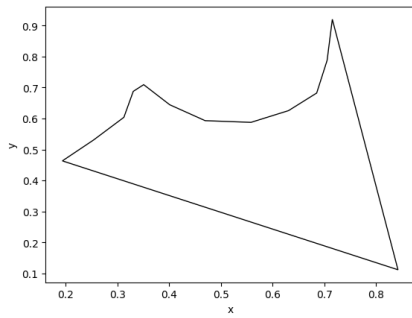
Rys. w11 Wielokąt



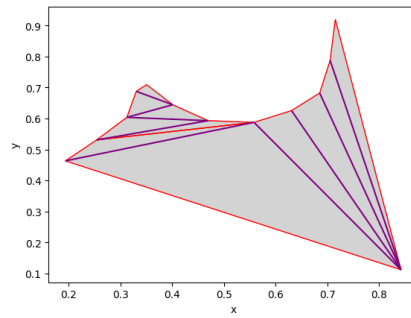
Rys. p11 Triangulacja podział



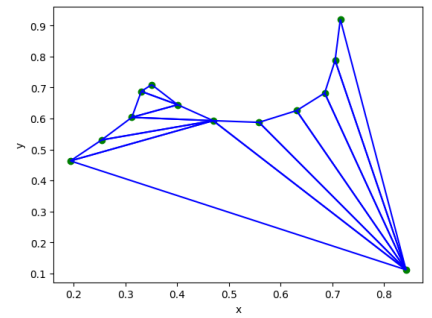
Rys. d11 Triangulacja Delaunay'a



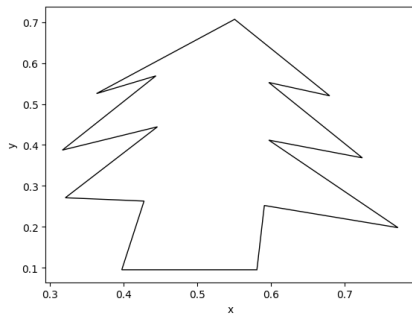
Rys. w12 Wielokąt



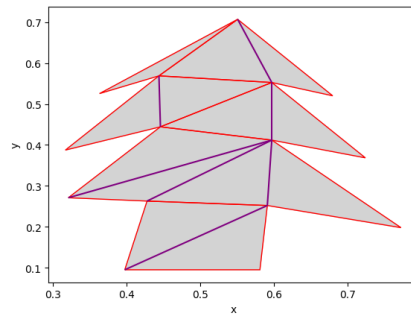
Rys. p12 Triangulacja podział



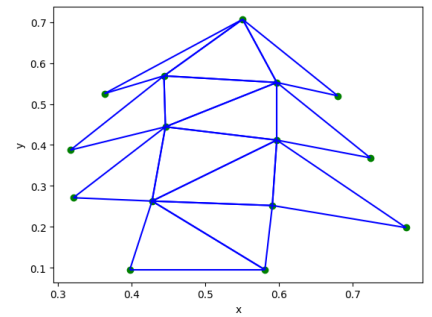
Rys. d12 Triangulacja Delaunay'a



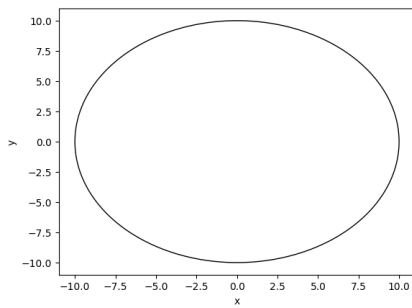
Rys. w13 Wielokąt



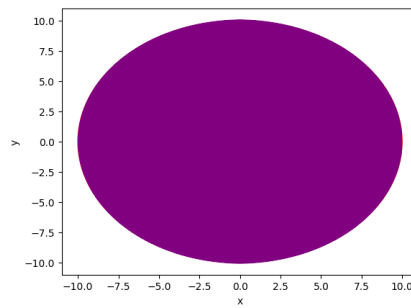
Rys. p13 Triangulacja podział



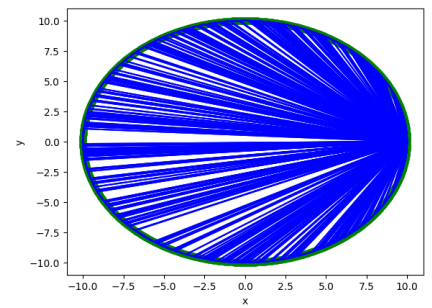
Rys. d13 Triangulacja Delaunay'a



Rys. wc Wielokąt

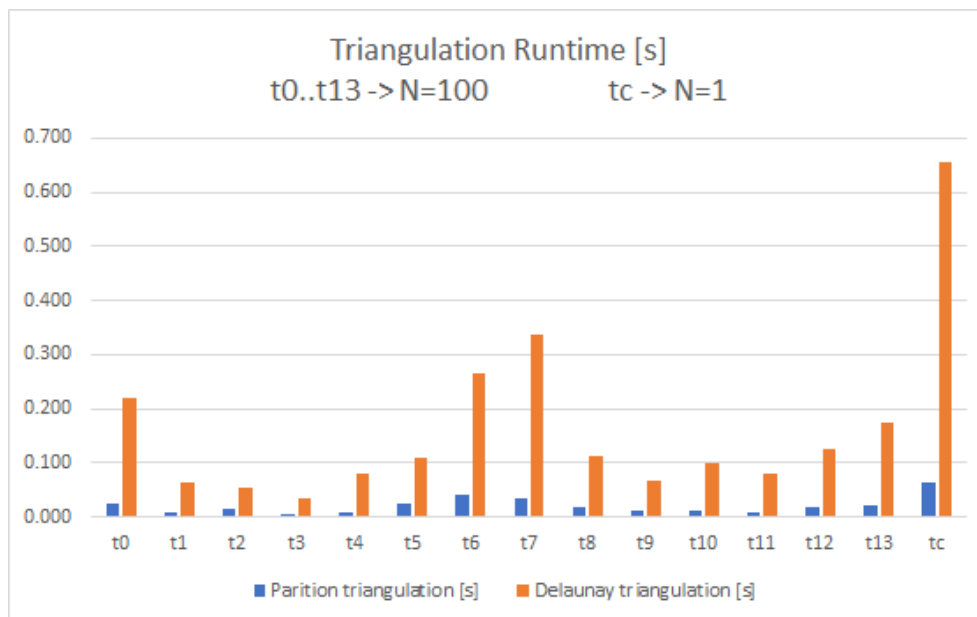


Rys. pc Triangulacja podział



Rys. dc Triangulacja Delaunay'a

Porównanie czasu działania



Rys. 5: Czas działania dla N powtórzeń testów z obrazków powyżej.

Test ID	Runs	Parition triangulation	Delaunay triangulation
t0	100	0.025s	0.221s
t1	100	0.009s	0.062s
t2	100	0.014s	0.053s
t3	100	0.006s	0.035s
t4	100	0.008s	0.081s
t5	100	0.026s	0.110s
t6	100	0.041s	0.266s
t7	100	0.033s	0.337s
t8	100	0.018s	0.114s
t9	100	0.013s	0.066s
t10	100	0.012s	0.100s
t11	100	0.009s	0.079s
t12	100	0.019s	0.125s
t13	100	0.021s	0.175s
tc	1	0.065s	0.655s

Tabela 1: Czas działania algorytmów triangulacji.